

Gradient of Vector Valued Functions and Neural Networks

Lecture Notes

April 28, 2026

1 Gradient of Vector Valued Functions or Jacobian Matrix

We know the definition of partial derivative of a scalar valued function. If $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a vector valued function where $n, m \geq 1$ and $\mathbf{x} \in \mathbb{R}^n$, then $f(\mathbf{x})$ can be written as:

$$f(\mathbf{x}) = (f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x}))$$

where each $f_i(\mathbf{x})$ denotes the i^{th} component of $f(\mathbf{x})$ for $1 \leq i \leq m$. We had used column vector notation for vectors so far:

$$\mathbf{x} = (x_1, x_2, \dots, x_n) = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = [x_1, x_2, \dots, x_n]^T$$

Thus, $\mathbb{R}^n \cong M_{n \times 1}(\mathbb{R})$. Further,

$$f(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix} = [f_1(\mathbf{x}), \dots, f_m(\mathbf{x})]^T \in \mathbb{R}^m$$

and $f = [f_1, f_2, \dots, f_m]^T$.

1.1 Definition: Gradient of a Vector Valued Function

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $\mathbf{x} \in \mathbb{R}^n$. We define:

$$\nabla_{\mathbf{x}} f = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

This matrix is called the **gradient of f** or the **Jacobian of f** with respect to $\mathbf{x}$. For $\mathbf{a} \in \mathbb{R}^n$, $\nabla_{\mathbf{x}} f(\mathbf{a}) = \left[\frac{\partial f_i}{\partial x_j}(\mathbf{a}) \right]_{i,j}$.

1.2 Jacobian appears naturally in chain rule!

Recall the chain rule. If $w = f(x_1, x_2, x_3)$ is a scalar function in $D \subseteq \mathbb{R}^3$ which is $C^1(D)$ (i.e., all first-order partials exist in D and are continuous in D) and x_1, x_2, x_3 are themselves functions of u_1, u_2 , i.e.,

$$x_1 = x_1(u_1, u_2), \quad x_2 = x_2(u_1, u_2), \quad x_3 = x_3(u_1, u_2)$$

each of which satisfies $x_i \in C^1(B)$ where B is in the $u_1 u_2$ -plane. Where if $(u_1, u_2) \in B$, then $(x_1, x_2, x_3) \in D$. Then the partial derivatives of w w.r.t. u_1 and u_2 can be written in terms of x_1, x_2, x_3 as:

$$\begin{aligned} \frac{\partial w}{\partial u_1} &= \frac{\partial w}{\partial x_1} \frac{\partial x_1}{\partial u_1} + \frac{\partial w}{\partial x_2} \frac{\partial x_2}{\partial u_1} + \frac{\partial w}{\partial x_3} \frac{\partial x_3}{\partial u_1} \\ \frac{\partial w}{\partial u_2} &= \frac{\partial w}{\partial x_1} \frac{\partial x_1}{\partial u_2} + \frac{\partial w}{\partial x_2} \frac{\partial x_2}{\partial u_2} + \frac{\partial w}{\partial x_3} \frac{\partial x_3}{\partial u_2} \end{aligned}$$

In matrix form:

$$\begin{bmatrix} \frac{\partial w}{\partial u_1} & \frac{\partial w}{\partial u_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial w}{\partial x_1} & \frac{\partial w}{\partial x_2} & \frac{\partial w}{\partial x_3} \end{bmatrix} \begin{bmatrix} \frac{\partial x_1}{\partial u_1} & \frac{\partial x_1}{\partial u_2} \\ \frac{\partial x_2}{\partial u_1} & \frac{\partial x_2}{\partial u_2} \\ \frac{\partial x_3}{\partial u_1} & \frac{\partial x_3}{\partial u_2} \end{bmatrix}$$

To see the chain rule as matrix multiplication, it is natural to see gradients as row vectors instead of column vectors. Accordingly, this can be expressed as:

$$\nabla_{\mathbf{u}} w = \nabla_{\mathbf{x}} w \nabla_{\mathbf{u}} \mathbf{h}$$

$$\underbrace{(1 \times 2)}_{\text{Result}} = \underbrace{(1 \times 3)}_{\text{Scalar Grad}} \cdot \underbrace{(3 \times 2)}_{\text{Jacobian}}$$

where $\mathbf{h}(u_1, u_2) = (x_1, x_2, x_3)$. The matrix $\nabla_{\mathbf{u}} \mathbf{h}$ is known as Jacobian of $\mathbf{h}$ with respect to $\mathbf{u}$.

2 Examples

Example 1

Find the gradient of $f(\mathbf{x}) = A\mathbf{x}$ where $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $A \in M_{m \times n}(\mathbb{R})$.

Solution: To compute $\nabla_{\mathbf{x}} f$, we need to know $\frac{\partial f_i}{\partial x_j}$ where $f = [f_1, \dots, f_m]^T$ and $\mathbf{x} = [x_1, \dots, x_n]^T$.

$$f(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix} = A\mathbf{x} = \begin{bmatrix} A_{11} & \dots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \dots & A_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

$\Rightarrow f_i(\mathbf{x}) = \sum_{j=1}^n A_{ij} x_j$. Therefore, $\frac{\partial f_i}{\partial x_j} = A_{ij}$ for all $1 \leq i \leq m$ and $1 \leq j \leq n$.

$$\therefore \nabla_{\mathbf{x}} f = \begin{bmatrix} A_{11} & \dots & A_{1n} \\ \vdots & \ddots & \vdots \\ A_{m1} & \dots & A_{mn} \end{bmatrix} = A$$

Thus, $\nabla_{\mathbf{x}} f$ evaluated at any $\mathbf{a} \in \mathbb{R}^n$ is A itself. Note (Exercise): If $f(\mathbf{x}) = A\mathbf{x} + \mathbf{b}$ where $\mathbf{b} \in \mathbb{R}^m$ is constant, $\nabla_{\mathbf{x}} f = A$.

3 Supervised Learning Problem

Suppose you have m training examples (x^i, y^i) for $1 \leq i \leq m$ where $x^i \in \mathbb{R}^n$. For example:

Ex	Living area (x_1)	No. of bedrooms (x_2)	...	Price (y)
1	x_1^1	x_2^1	...	y^1
2	x_1^2	x_2^2	...	y^2
$\vdots$	$\vdots$	$\vdots$	...	$\vdots$
m	x_1^m	x_2^m	...	y^m

We want to find a nice function $h : \mathbb{R}^n \rightarrow \mathbb{R}$ such that $h(\mathbf{x})$ is a good predictor for the corresponding value y . These problems are called regression problems. In linear regression, we approximate y using a linear function of $\mathbf{x}$:

$$h(\mathbf{x}) = h(x_1, \dots, x_n) = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

where θ_j are called parameters or weights. Note that $h(\mathbf{x}) = \mathbf{x}^T \theta$ (seeing $\mathbf{x}$ and θ as column vectors).

For each example i : $h(x^i) = (x^i)^T \theta$. The error for example i is $y^i - h_\theta(x^i)$. Let:

$$e(\theta) = \begin{bmatrix} y^1 - h_\theta(x^1) \\ \vdots \\ y^m - h_\theta(x^m) \end{bmatrix} = \begin{bmatrix} y^1 - (x^1)^T \theta \\ \vdots \\ y^m - (x^m)^T \theta \end{bmatrix} = \mathbf{y} - \Phi \theta$$

where $\Phi \in M_{m \times n}(\mathbb{R})$ is the input feature matrix. We aim to minimize the error function $L(\theta) = \|e(\theta)\|^2$, i.e., find θ such that $L(\theta)$ is minimum.

Example 2: Application of Jacobian in Machine Learning (Supervised Learning)

Let us consider the linear model $\hat{\mathbf{y}} = \Phi \theta$ to predict y , where $\theta \in \mathbb{R}^D \cong M_{D \times 1}(\mathbb{R})$ is a parameter vector and $\Phi \in M_{N \times D}(\mathbb{R})$ are input features. Let $e(\theta) = \hat{\mathbf{y}} - \Phi \theta$ and $L(\theta) = \|e\|^2$. Find $\nabla_\theta L$.

Solution: $\hat{\mathbf{y}} \in \mathbb{R}^N$ and hence $e(\theta) \in \mathbb{R}^N$. We have the composition $\mathbb{R}^D \xrightarrow{e} \mathbb{R}^N \xrightarrow{L} \mathbb{R}$. $\theta \mapsto y - \Phi \theta \mapsto \|y - \Phi \theta\|^2$. By the chain rule: $\nabla_\theta L = \nabla_e L \cdot \nabla_\theta e$. $\nabla_\theta e$ is the Jacobian of the vector valued function e . Since $e(\theta) = y - \Phi \theta$ is a linear function, from the previous example, $\nabla_\theta e = -\Phi$. Now, $L(e_1, \dots, e_N) = e_1^2 + e_2^2 + \dots + e_N^2$.

$$\begin{aligned} \nabla_e L &= \left[\frac{\partial L}{\partial e_1} \quad \dots \quad \frac{\partial L}{\partial e_N} \right] = [2e_1 \quad \dots \quad 2e_N] = 2\mathbf{e}^T = 2(\mathbf{y} - \Phi \theta)^T = 2(\mathbf{y}^T - \theta^T \Phi^T) \\ &\Rightarrow \nabla_\theta L = -2(\mathbf{y}^T - \theta^T \Phi^T) \Phi \end{aligned}$$

4 Gradients of Matrices

Suppose we have two matrices $A \in M_{m \times n}(\mathbb{R})$ and $B \in M_{p \times q}(\mathbb{R})$ and wish to compute the gradient of A w.r.t. B . Think of B as an independent variable and A being dependent on B ($A = f(B)$). A has mn component functions $A_{11}, \dots, A_{mn}$, each being a function of pq many variables $B_{11}, \dots, B_{pq}$. There are two definitions that are prevalent. First is

4.1 Tensor Notation

Consider the first component function of A , i.e., A_{11} , seen as a scalar valued function with B as its input. Define

$$\nabla_B A_{11} := \begin{bmatrix} \frac{\partial A_{11}}{\partial B_{11}} & \frac{\partial A_{11}}{\partial B_{12}} & \cdots & \frac{\partial A_{11}}{\partial B_{1q}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial A_{11}}{\partial B_{p1}} & \frac{\partial A_{11}}{\partial B_{p2}} & \cdots & \frac{\partial A_{11}}{\partial B_{pq}} \end{bmatrix}$$

Similarly, we can define the gradient for each of the component functions A_{i1} where i is varied. This would give us a 3 dimensional array. Finally, for each fixed i , upon varying j in A_{ij} , the gradient of A can now be defined and it would be a four-dimensional array called a tensor whose entries are $J_{ijkl} = \frac{\partial A_{ij}}{\partial B_{kl}}$.

4.2 Second Definition

In practical applications, it is often desirable to reshape the matrix into a vector and continue working with this Jacobian matrix. Since $M_{m \times n}(\mathbb{R}) \cong \mathbb{R}^{mn}$, we see that in $A = f(B)$, f is equivalently a function $f : \mathbb{R}^{pq} \rightarrow \mathbb{R}^{mn}$. Matrix is reshaped into a vector $\tilde{B} \in \mathbb{R}^{pq}$. We see this definition of gradient in an example.

Example 1: Gradient of vectors w.r.t. matrices

Consider the map $A \xrightarrow{f} A\mathbf{x}$ where $A \in M_{M \times N}(\mathbb{R})$ and $\mathbf{x} \in \mathbb{R}^N$.

1. What is the dimension of $\nabla_A f$?
2. Compute the gradient of f w.r.t. A .

Solution: Since $A\mathbf{x} \in \mathbb{R}^M$, $f : \mathbb{R}^{MN} \rightarrow \mathbb{R}^M$. Thus $\nabla_A f \in M_{M \times (MN)}$. Let $f_1(A), \dots, f_M(A)$ be component functions. To compute the Jacobian matrix, we view $\mathbf{A}$ as a vector in $\mathbb{R}^{MN}$:

$$\text{vec}(\mathbf{A}) = [A_{11}, A_{12}, \dots, A_{1N}, A_{21}, \dots, A_{2N}, \dots, A_{M1}, \dots, A_{MN}]^T$$

We need $\frac{\partial f_i}{\partial A_{kj}}$. Note that f_i , i.e., the i -th entry of $A\mathbf{x}$, denoted $(A\mathbf{x})_i$ is given by

$$f_i = (A\mathbf{x})_i = \sum_{l=1}^N A_{il}x_l.$$

$$\frac{\partial (A\mathbf{x})_i}{\partial A_{kj}} = \begin{cases} x_j & \text{if } k = i \\ 0 & \text{if } k \neq i \end{cases}$$

For the first row ($i = 1$):

The derivatives are $\frac{\partial f_1}{\partial A_{11}} = x_1, \frac{\partial f_1}{\partial A_{12}} = x_2, \dots, \frac{\partial f_1}{\partial A_{1N}} = x_N$. All other derivatives for f_1 (with respect to rows 2 through M) are zero. This forms the row vector:

$$[x_1, x_2, \dots, x_N, 0, 0, \dots, 0] = [\mathbf{x}^T, \mathbf{0}, \dots, \mathbf{0}]$$

For the k^{th} row ($i = k$):

The partial derivatives are zero for all A_{ij} where $i \neq k$. The non-zero derivatives occur only in the block corresponding to the k^{th} row of $\mathbf{A}$. The resulting Jacobian $\nabla_A f$ is a block matrix:

$$\nabla_A f = \begin{bmatrix} x_1 \dots x_N & 0 \dots 0 & \dots & 0 \dots 0 \\ 0 \dots 0 & x_1 \dots x_N & \dots & 0 \dots 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 \dots 0 & 0 \dots 0 & \dots & x_1 \dots x_N \end{bmatrix}_{M \times (MN)}$$

Example 2: Gradient of matrices w.r.t. matrices

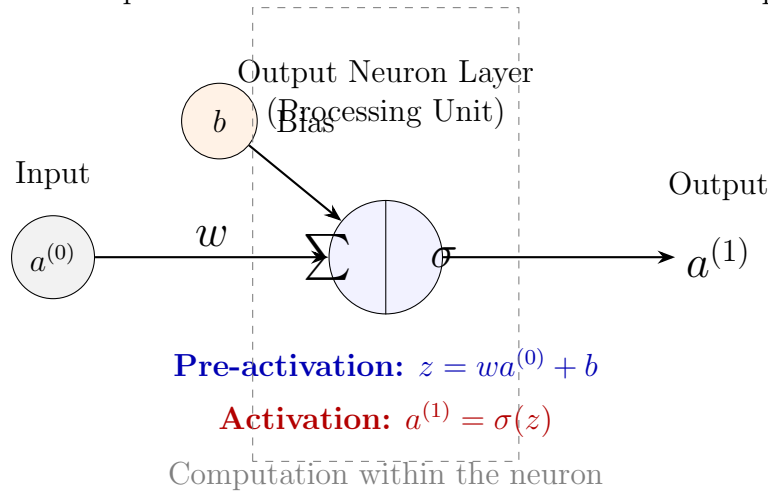
Let $A \in M_{M \times N}(\mathbb{R})$ and $f(A) = A^T A = K$. What is $\nabla_A K$? $K \in M_{N \times N}(\mathbb{R})$. $K_{pq} = (A^T A)_{pq} = \sum_{m=1}^M (A^T)_{pm} A_{mq} = \sum_{m=1}^M A_{mp} A_{mq}$.

$$\frac{\partial K_{pq}}{\partial A_{ij}} = \begin{cases} 2A_{ip} & \text{if } p = q \text{ and } j = p \\ A_{iq} & \text{if } p \neq q \text{ but } j = p \\ A_{ip} & \text{if } p \neq q \text{ but } j = q \\ 0 & \text{otherwise} \end{cases}$$

Note on Convention Change: From this point onwards, following the convention for Neural Networks, the symbol $\frac{\partial}{\partial x}$ is used instead of $\nabla_{\mathbf{x}}$. Recall also that gradients are now considered as **row vectors**.

5 Artificial Neural Network

A neural network is fundamentally a mathematical function that takes an input $\mathbf{X}$ and gives an output $\mathbf{Y}$. Circles denote neurons and lines correspond to connections.



5.1 Basic Model

Define $a^{(1)} = \sigma(wa^{(0)} + b)$ where $b, w \in \mathbb{R}$ and σ is an activation function (e.g., hyperbolic tangent $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ or sigmoid $f(x) = \frac{1}{1+e^{-x}}$). Neurons interact and get "activated" when they exceed a threshold.

5.2 General Single-Layered Neural Network

For a neural network with n input neurons and m output neurons, the transformation is defined by the weight matrix $W^{(1)}$ and the bias vector $b^{(1)}$.

1. The Weight Matrix $W^{(1)}$

The weight matrix $W^{(1)}$ has dimensions $m \times n$, where the element $w_{ij}^{(1)}$ represents the weight of the connection between the j^{th} input neuron and the i^{th} output neuron.

$$W^{(1)} = \begin{bmatrix} w_{00}^{(1)} & w_{01}^{(1)} & \dots & w_{0,n-1}^{(1)} \\ w_{10}^{(1)} & w_{11}^{(1)} & \dots & w_{1,n-1}^{(1)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m-1,0}^{(1)} & w_{m-1,1}^{(1)} & \dots & w_{m-1,n-1}^{(1)} \end{bmatrix}_{m \times n}$$

2. The Bias Vector $b^{(1)}$

The bias vector $b^{(1)}$ is an m -dimensional column vector, with one bias term for each output neuron.

$$\vec{b}^{(1)} = \begin{bmatrix} b_0^{(1)} \\ b_1^{(1)} \\ \vdots \\ b_{m-1}^{(1)} \end{bmatrix}_{m \times 1}$$

3. Input and Output Vectors

The input activation vector $\vec{a}^{(0)}$ and the resulting output activation vector $\vec{a}^{(1)}$ are defined as follows:

$$\vec{a}^{(0)} = \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_{n-1}^{(0)} \end{bmatrix}_{n \times 1} \quad \text{and} \quad \vec{a}^{(1)} = \begin{bmatrix} a_0^{(1)} \\ a_1^{(1)} \\ \vdots \\ a_{m-1}^{(1)} \end{bmatrix}_{m \times 1}$$

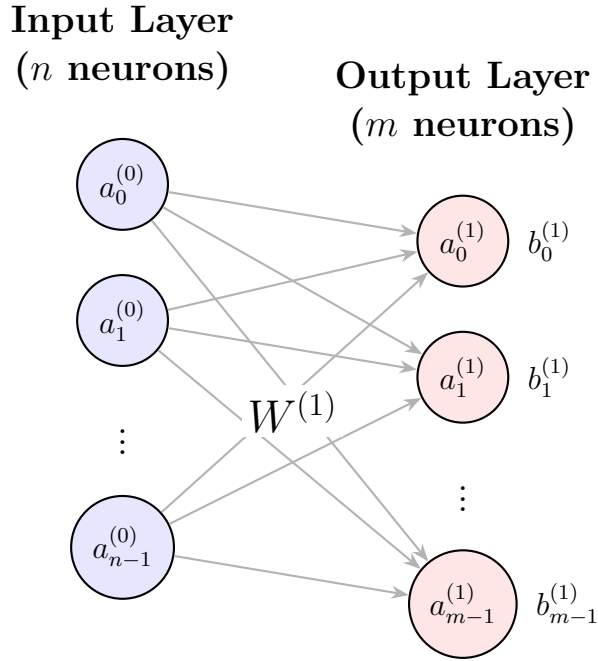
4. The Forward Propagation Equation

The relationship between these components is given by the equation:

$$\vec{a}^{(1)} = \sigma \left(W^{(1)} \cdot \vec{a}^{(0)} + \vec{b}^{(1)} \right)$$

Where:

- $W^{(1)} \cdot \vec{a}^{(0)} + \vec{b}^{(1)} = \vec{z}^{(1)}$ is the **pre-activation vector**.
- σ is the element-wise **activation function** (e.g., Sigmoid or Tanh).



$$\vec{a}^{(1)} = \sigma \left(\mathbf{W}^{(1)} \vec{a}^{(0)} + \vec{b}^{(1)} \right)$$

Where $\mathbf{W}^{(1)}$ is an $m \times n$ weight matrix
and $\vec{b}^{(1)}$ is an $m \times 1$ bias vector.

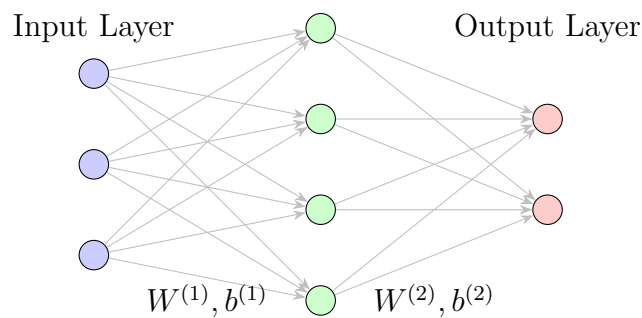
6 Multi-Layer Networks

In a multi-layered network:

$$\mathbf{a}^{(L)} = \sigma(W^{(L)} \mathbf{a}^{(L-1)} + \mathbf{b}^{(L)})$$

These are called "Feed Forward" neural networks.

Hidden Layer



7 Training a NOT Function

Problem: Train a neuron to simulate a **NOT** gate.

- If $a^{(0)} = 1 \implies a^{(1)} \approx 0$

- If $a^{(0)} = 0 \implies a^{(1)} \approx 1$

Using $\sigma(x) = \tanh(x)$ and model $a^{(1)} = \sigma(wa^{(0)} + b)$.

Options: (a) $w = 10, b = 0$ (b) $w = -3, b = 0$ (c) $w = 3, b = 1$ (d) $w = -5, b = 5$

Solution: Test option (d): If $a^{(0)} = 1 \implies z = -5(1) + 5 = 0 \implies \tanh(0) = 0$. (Correct)

If $a^{(0)} = 0 \implies z = -5(0) + 5 = 5 \implies \tanh(5) \approx 1$. (Correct)

Persuading a neural network to do tasks (like image recognition etc) is just a job of teaching it the right weights and biases.

8 The Forward Propagation (The Predictive Pass)

Forward propagation is the process of calculating the output of the neural network by passing the input through successive layers.

8.1 Step-by-Step Layer Transformation

For any layer l (where $l = 1$ is the first hidden layer and $l = L$ is the output layer):

1. **(Pre-activation):** We compute the vector $\mathbf{z}^{(l)}$. This represents the signal before the non-linear "firing" occurs.

$$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

The values of $\mathbf{z}^{(l)}$ must be cached (stored) during this pass because they are required to calculate the derivatives of the activation function (σ') during the backward pass.

2. **Non-linear Activation:** We apply the activation function σ element-wise.

$$\mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)})$$

8.2 The Objective (Cost Function)

Once the final activation $\mathbf{a}^{(L)}$ is reached, we calculate the error relative to the target $\mathbf{y}$ using a Cost Function C :

$$C = \frac{1}{2} \|\mathbf{a}^{(L)} - \mathbf{y}\|^2 = \frac{1}{2} \sum_i (a_i^{(L)} - y_i)^2$$

9 The Backward Propagation (The Learning Pass)

The goal of backpropagation is to calculate the partial derivatives $\frac{\partial C}{\partial W^{(l)}}$ and $\frac{\partial C}{\partial \mathbf{b}^{(l)}}$ so that the weights and biases can be updated using Gradient Descent.

9.1 The Engine: The Chain Rule

To find how the Cost C changes with respect to a weight in layer l , we follow the dependency chain backwards:

$$C \rightarrow \mathbf{a}^{(L)} \rightarrow \mathbf{z}^{(L)} \rightarrow \dots \rightarrow \mathbf{a}^{(l)} \rightarrow \mathbf{z}^{(l)} \rightarrow W^{(l)}$$

Using the Chain Rule (in row-vector form):

$$\frac{\partial C}{\partial W^{(l)}} = \frac{\partial C}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l)}}{\partial W^{(l)}}$$

Backpropagation Basics

Recall that the feed-forward equation at the n^{th} layer is given by:

$$a^{(n)} = \sigma(z^{(n)})$$

where the **pre-activation variable** $z^{(n)}$ is defined as:

$$z^{(n)} = W^{(n)}a^{(n-1)} + b^{(n)}$$

The variable $z^{(n)}$ represents the state of the neuron after the linear transformation but before the non-linear activation function is applied.

- **Computational Efficiency:** During the forward pass, we store (cache) $z^{(n)}$. This is essential because the backward pass requires the derivative of the activation function, $\sigma'(z^{(n)})$.
- **Avoiding Redundancy:** By saving $z^{(n)}$, we avoid re-performing expensive matrix-vector multiplications and additions during the backpropagation step.

The objective is to minimize the **Cost Function** C . For a single training example, we define:

$$C = \frac{1}{2} \|a^{(N)} - y\|^2, \quad 0 \leq n \leq N$$

To update weights W and biases b , we need to determine the gradients $\frac{\partial C}{\partial W}$ and $\frac{\partial C}{\partial b}$.

Some extra informations:(Backpropagation)

Calculating the total derivative of the cost function with respect to a weight in an early layer is computationally prohibitive if done from scratch. Backpropagation provides:

- **Modularity:** Each layer only computes its local gradient and passes it backward.
- **Linear Complexity:** The computation scales linearly with the number of layers $O(N)$, whereas a brute-force derivative calculation would scale quadratically $O(N^2)$.

Example Architecture: Consider a network with an input layer $a^{(0)}$, one hidden layer $a^{(1)}$, and an output layer $a^{(2)}$.

$$a^{(2)} = \sigma(z^{(2)}) \quad \text{and} \quad a^{(1)} = \sigma(z^{(1)})$$

Using the chain rule in **row-vector convention**:

$$\frac{\partial C}{\partial W^{(2)}} = \left(\frac{\partial C}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \right) \cdot \frac{\partial z^{(2)}}{\partial W^{(2)}} \quad (1)$$

$$\frac{\partial C}{\partial W^{(1)}} = \left(\frac{\partial C}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \right) \cdot \frac{\partial z^{(2)}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial W^{(1)}} \quad (2)$$

Notice that the term in the parentheses $(\cdot)$ in Eq. (1) is reused in Eq. (2). This reuse of computations is what makes Backpropagation efficient.

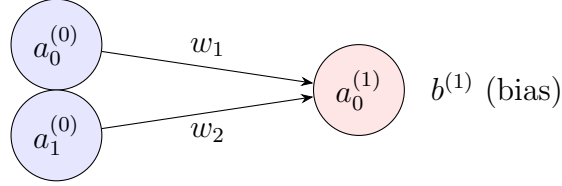
Generalized Backpropagation

For a network with N layers, the gradient calculation for the i^{th} layer can be generalized as:

$$\frac{\partial C}{\partial W^{(i)}} = \frac{\partial C}{\partial a^{(N)}} \cdot \frac{\partial a^{(N)}}{\partial z^{(N)}} \cdot \frac{\partial z^{(N)}}{\partial a^{(N-1)}} \cdots \frac{\partial a^{(i+1)}}{\partial z^{(i+1)}} \cdot \frac{\partial z^{(i+1)}}{\partial a^{(i)}} \cdot \frac{\partial a^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial W^{(i)}}$$

Problem 1: Numerical Exercise

Consider the following neural network with a single training example:



Given:

- Activation function: $\sigma(x) = \frac{1}{1+e^{-x}}$ (Sigmoid)
- Learning rate: $\alpha = 0.1$
- Input: $X = (0.8, 0.6)$, Target: $y = 1.0$
- Initial parameters: $w_1 = 0.3, w_2 = 0.5, b = 0.1$

Task: Perform one step of Forward Propagation (FP) and Backpropagation (BP).

Derivative of Sigmoid:

$$\sigma'(x) = \frac{d}{dx}(1 + e^{-x})^{-1} = -(1 + e^{-x})^{-2} \cdot (-e^{-x}) = \frac{e^{-x}}{(1 + e^{-x})^2} = \sigma(x)(1 - \sigma(x))$$

Step 1: Forward Propagation

First, calculate the pre-activation $z^{(1)}$:

$$z^{(1)} = w_1 a_0^{(0)} + w_2 a_1^{(0)} + b = (0.3)(0.8) + (0.5)(0.6) + 0.1 = 0.24 + 0.30 + 0.1 = 0.64$$

Now, calculate the activation $a^{(1)}$:

$$a^{(1)} = \sigma(0.64) = \frac{1}{1 + e^{-0.64}} \approx 0.6547$$

Calculate the Cost C :

$$C = \frac{1}{2}(a^{(1)} - y)^2 = \frac{1}{2}(0.6547 - 1)^2 = \frac{1}{2}(-0.3453)^2 \approx 0.0596$$

Step 2: Backpropagation

Update rule for weights: $w^{\text{new}} = w^{\text{old}} - \alpha \frac{\partial C}{\partial w}$. Using the chain rule for w_1 :

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial w_1}$$

- $\frac{\partial C}{\partial a^{(1)}} = (a^{(1)} - y) = 0.6547 - 1 = -0.3453$
- $\frac{\partial a^{(1)}}{\partial z^{(1)}} = \sigma'(0.64) = \frac{e^{-0.64}}{(1+e^{-0.64})^2} \approx 0.226$
- $\frac{\partial z^{(1)}}{\partial w_1} = a_0^{(0)} = 0.8$

Substituting the values:

$$\frac{\partial C}{\partial w_1} = (-0.3453) \times (0.226) \times (0.8) = -0.0624$$

Similarly for w_2 :

$$\frac{\partial C}{\partial w_2} = \frac{\partial C}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot a_1^{(0)} = (-0.3453) \times (0.226) \times (0.6) = -0.047$$

The updated weight vector [as per the notation in Page 6](#) is:

$$W^{\text{new}} = \begin{bmatrix} 0.3 & 0.5 \end{bmatrix} - 0.1 \begin{bmatrix} -0.0624 & -0.047 \end{bmatrix} = \begin{bmatrix} 0.30624 & 0.5047 \end{bmatrix}$$

Step 3: Updating the Bias

The gradient for the bias $b^{(1)}$ is:

$$\frac{\partial C}{\partial b^{(1)}} = \frac{\partial C}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial b^{(1)}}$$

Since $\frac{\partial z^{(1)}}{\partial b^{(1)}} = 1$:

$$\frac{\partial C}{\partial b^{(1)}} = (-0.3453) \times (0.226) \times (1) = -0.078$$

The updated bias is:

$$b^{\text{new}} = b^{\text{old}} - \alpha \frac{\partial C}{\partial b^{(1)}} = 0.1 - 0.1(-0.078) = 0.1078$$

Summary Table (End of Step 1)

$b^{(1)}$	$W^{(1)}$	$a^{(1)}$	C	$\nabla_{W^{(1)}} C$	$\frac{\partial C}{\partial b^{(1)}}$	$W^{(1),\text{new}}$	$b^{(1),\text{new}}$
0.1	$\begin{bmatrix} 0.3 & 0.5 \end{bmatrix}$	0.655	0.0596	$\begin{bmatrix} -0.0624 & -0.047 \end{bmatrix}$	-0.078	$\begin{bmatrix} 0.3062 & 0.5047 \end{bmatrix}^T$	0.1078

Second Step (Exercise)

Continuing with the updated values for the next iteration:

$b^{(1)}$	$W^{(1)}$	$a^{(1)}$	C	$\nabla_{W^{(1)}} C$	$\frac{\partial C}{\partial b^{(1)}}$	$W^{(1),\text{new}}$	$b^{(1),\text{new}}$
0.1078	$\begin{bmatrix} 0.3062 & 0.5047 \end{bmatrix}$	0.6583	0.0584	$\begin{bmatrix} -0.0615 & -0.0461 \end{bmatrix}$	-0.0768	$\begin{bmatrix} 0.3124 & 0.5093 \end{bmatrix}$	0.1155

Problem 2

Consider a single-layer neural network with **3 input neurons** and **1 output neuron**. Perform one full iteration of training (Forward and Backward propagation) using the following specifications:

- **Inputs:** $\mathbf{x} = [0.5, -0.2, 0.1]^T$
- **Target:** $y = 1.0$
- **Activation Function:** $\sigma(z) = \tanh(z)$
- **Learning Rate:** $\alpha = 0.4$
- **Initial Parameters:** $w_1 = 0.2, w_2 = -0.3, w_3 = 0.4$ and $b = 0.1$

Step 1: Forward Propagation

1. **Linear Combination (Pre-activation z):**

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

$$z = (0.2)(0.5) + (-0.3)(-0.2) + (0.4)(0.1) + 0.1$$

$$z = 0.1 + 0.06 + 0.04 + 0.1 = \mathbf{0.3000}$$

2. **Activation (a):**

$$a = \tanh(0.3) \approx \mathbf{0.2913}$$

3. **Cost (C):**

$$C = \frac{1}{2}(a - y)^2 = \frac{1}{2}(0.2913 - 1.0)^2 \approx \mathbf{0.2511}$$

Step 2: Backward Propagation

1. **Derivative of Activation:** For $\tanh(z)$, $\sigma'(z) = 1 - \tanh^2(z) = 1 - a^2$.

$$\sigma'(0.3) = 1 - (0.2913)^2 \approx \mathbf{0.9151}$$

2. **Gradient of Cost w.r.t Pre-activation, say δ :**

$$\begin{aligned} \frac{\partial C}{\partial a} \cdot \frac{\partial a}{\partial z} &= (a - y) \cdot \sigma'(z) \\ &= (0.2913 - 1.0) \cdot 0.9151 = -0.7087 \cdot 0.9151 \approx \mathbf{-0.6485} \end{aligned}$$

3. **Parameter Gradients:**

- $\frac{\partial C}{\partial w_1} = \delta \cdot x_1 = -0.6485 \cdot 0.5 = \mathbf{-0.3243}$
- $\frac{\partial C}{\partial w_2} = \delta \cdot x_2 = -0.6485 \cdot (-0.2) = \mathbf{0.1297}$
- $\frac{\partial C}{\partial w_3} = \delta \cdot x_3 = -0.6485 \cdot 0.1 = \mathbf{-0.0649}$
- $\frac{\partial C}{\partial b} = \delta \cdot 1 = \mathbf{-0.6485}$

Step 3: Parameter Update

Using the rule $\theta_{new} = \theta_{old} - \alpha \frac{\partial C}{\partial \theta}$:

- $w_1^{new} = 0.2 - 0.4(-0.3243) = \mathbf{0.3297}$
- $w_2^{new} = -0.3 - 0.4(0.1297) = \mathbf{-0.3519}$
- $w_3^{new} = 0.4 - 0.4(-0.0649) = \mathbf{0.4260}$
- $b^{new} = 0.1 - 0.4(-0.6485) = \mathbf{0.3594}$

Summary Table (End of Step 1)

$b^{(1)}$	$W^{(1)}$	$a^{(1)}$	C	$\nabla_{W^{(1)}} C$	$\frac{\partial C}{\partial b^{(1)}}$	$W^{(1),new}$	$b^{(1),new}$
0.1	$\begin{bmatrix} 0.2 \\ -0.3 \\ 0.4 \end{bmatrix}^T$	0.2913	0.2511	$\begin{bmatrix} -0.3243 \\ 0.1297 \\ -0.0649 \end{bmatrix}^T$	-0.6485	$\begin{bmatrix} 0.3297 \\ -0.3519 \\ 0.4260 \end{bmatrix}^T$	0.3594